2026 年第九届传智杯全国大学生数智创新与 AI 应用大赛 · AI 创新应用挑战赛（个人赛）

Prompt 历史快照与迭代记录

——低卡轻食点餐系统的 AI 协作过程留档

参赛者：as5827

代码仓库：https://github.com/as5827/dika-light-meal-miniprogram · 2026 年 10 月

本册收录该项目开发过程中的三份对话快照与六份迭代记录。快照保留当时的报错原文、执行命令与判断顺序，用于说明问题如何被定位；迭代记录按「需求—拆解—取舍—接口—验证—没做的」组织，逐条说明每次功能落地的依据。所有内容均为纪要（节选），非逐字转录。

第一部分 对话快照

对话快照 01 · 从建仓到第一次成功推送

> 说明：以下是当时的对话纪要（节选，非逐字转录），保留了真实的报错、命令与判断顺序。截图/录屏可另存，本文件只保留可检索的文字。

背景

项目代码已经在本地跑通（H5 + Node 后端），要迁到 GitHub 上，让评审能公开访问。

我说的（原话）

1. 「我参与的是个人赛 你现在先把自己接上github」
2. 「加好了 用户名：grorge00 我使用的账号是grorge@qq.com 全局一键帮我替换成grorge@qq.com吧 我参加传智杯的项目名称是dika-light-meal-miniprogram 帮我解析上传一下主体代码吧」
3. 「为什么我没有在仓库看到我的项目？」
4. 「那换2977843600@qq.com这个账号你看可以不可以」

它先给出的判断

> "仓库已建，6 个提交已推送。"

结论下早了。三件事同时发生了，只报了一件。

实际卡住的四个环节

① 建仓接口 **400**

```
POST /user/repos → 400
{"message":"Problems parsing JSON"}
```

原因：PowerShell 5.1 把中文描述按 GBK 编码传给 `curl.exe`，GitHub 收到非法 UTF-8。

改法：把 JSON 写进 UTF-8 文件，用 `--data-binary @file`。一次通过，返回 201。

② **"Repository not found"** 不是权限问题

`git ls-remote` 报 not found，第一反应是权限，实际是仓库根本没创建成功（上一条的 400 就是根因）。

③ 换账号时撞上风控

```
ssh -T git@github.com      → Hi grorge00! You've successfully authenticated
GET /users/grorge00        → 404
GET /users/grorge00/repos  → 200 []
```

SSH 能认证、公开资料却 404，说明账号在库里存在但未公开（新号被标记）。这种号即使推上去，评委点链接也是 404。决定换号，改成 `as5827`（同一台机器上有它的凭据）。

④ **push** 反复被重置

```
fatal: unable to access 'https://github.com/...': Recv failure: Connection was reset
```

排查顺序：先怀疑仓库权限（不是）→ 再查连通性（`api.github.com` 通、`github.com` 超时）→ 最后发现系统里开着代理 `127.0.0.1:7897`，但 git 完全没走它。

```
git config http.proxy http://127.0.0.1:7897
git push → 成功
```

我做的三个决定

决定	依据
不追 grorge00，直接用 as5827	新号 404 意味着评委打不开，比赛场景下不能赌
不把 token 写进仓库或配置	只从系统凭据里读，用完即走；后来改成 Cookie 会话 + 环境变量
本地目录先迁到 Desktop 管理的新位置，再删旧的	两份代码必然改错，只留一份

结果

- 提交历史完整上线（先 6 条，后续迭代到 27 条）；
- 推送通道固定为「代理 + 系统凭据」，`git push` 不再需要额外参数；
- GitHub Desktop 接管本地目录，与远程保持同步。

复盘

这次折腾的教训不是"命令记错了"，而是结论下得太早：把"我这边 push 成功"当成了"评委能打开"。后面所有验收都改成用匿名接口复核（`private:false`、能否匿名读 README），而不是只看命令行回显。

对话快照 02 · 推荐引擎：从"能算出一套"到"算得站得住"

> 说明：对话纪要（节选）。这里的取舍不是一次成型，中间被推翻过一轮。

我说的（原话）

> 本项目为减脂餐创业商家打造轻量化点餐小程序……业务层面：面向小微轻食商家，提供低成本数字化接单方案，内置 **AI** 个性化减脂套餐推荐，结合用户身体指标给出适配餐品，同时双前端架构兼顾真实业务落地与评审演示便捷性。

它先给的方案

用大模型直接生成套餐，理由是这样"更像 AI"。

我否掉的原因写在 `prompts/03` 里：结果不可复现，评审问"为什么是这三道菜"时答不上来；而且每次调用都要钱、要网、要 key。改成本地营养学模型 + 组合搜索，大模型只负责把结论说顺口。

第一版跑出来的问题

女 28 岁 / 165cm / 58kg / 轻度活动 / 减脂：

```
目标：每日 1441 kcal、蛋白 93 g；单餐预算 576 kcal / 37 g
推荐：鸡胸肉藜麦沙拉 + 额外鸡胸肉 + 番茄时蔬汤
合计：600 kcal、蛋白 64 g
蛋白达成 173%
```

热量贴合，蛋白直接顶到目标的 1.7 倍。这不是"个性化"，是把所有能加蛋白的东西堆上去。

怎么改的

目标函数原本是：

```
score = 蛋白达成×0.5 − 热量偏差×0.35 + 价格分×0.15
```

蛋白项被 `min(1.6, p/target)` 截断在 1.6，等于告诉算法"蛋白越多越好"。加了过量惩罚：

```
overProtein = max(0, 蛋白 − 目标×1.3) / 目标
score = … − overProtein × 0.2
```

再跑：

```
合计 530 kcal、蛋白 59 g（目标 37 g，达成 159%）
```

仍然偏高，但这是有意的：轻食客群本来就冲高蛋白来的，压到 90% 反而推荐出"蔬菜沙拉 + 气泡水"这种没价值的组合。这个度是人工定的，不是算法自己选的。

第二轮问题：预算太狠

预算填 ¥10，第一版直接返回"无解"。改成两段式：

1. 先在预算内搜索；
2. 无解时退化为「满足营养下限（蛋白 ≥60%、热量 ≥70%）的最便宜组合」，并在结果里标 `budgetRelaxed: true`。

返回 ¥26 的全麦鸡胸三明治，前端明确写"预算内没有合适组合，已自动放宽"。

第三轮问题：营养数据从哪来

第一版想接公开营养库，但 28 道菜要逐条映射、还要处理"同菜不同做法"，成本和误差都不可控。最后：

- 每道菜补一个 `protein` 估算值；
- 在 `data/menu.json` 里写明 `nutritionNote: "protein 为估算值（g），仅用于推荐算法演示，非实验室检测数据"`；
- 文档里也标了。

宁可承认是估算，也不把假数据包装成检测报告。

验收

用例	结果
手算核对（女 28/165/58/轻度）	BMR 1310、TDEE 1801，与公式手算一致
素食过滤	三项全部素食
预算 ¥10	放宽后给最便宜合规组合，并标注
同参数两次请求	菜品 id 完全一致
非法参数	400 + 中文提示

复盘

"AI 推荐"最容易糊弄的地方是：随便给一套，说这是模型算的。要让它站得住，得能回答三个问题——输入是什么、怎么算的、为什么是这个结果。所以推荐日志里把 704 个候选、6ms 耗时、选中三道菜、偏差 2% 全打出来了（见 README「运行日志」）。

对话快照 03 · 让大模型只改语气，别动数字

> 说明：对话纪要（节选）。包含一次真实调用、一次兜底测试，以及"去 AI 味"的具体改法。

我说的（原话）

> API key:sk-*****（已脱敏） 调用一下试试并且再编辑润色文档和日志 尽量去除ai痕迹

两个要求：真的打通大模型，文档别写成一嘴 **AI** 腔。

第一步：先确认 key 能用

先不接业务，直接打 `POST /v1/chat/completions`：

```
HTTP 200 · model: deepseek-flash
choices[0].message.content: 减脂期保证蛋白质摄入，是因为……
```

能通。再接进 `/api/recommend` 的 `useAI` 分支。

第二步：约束它别乱动数字

推荐结果里全是算出来的数（BMR、目标热量、蛋白克数、偏差百分比），模型一旦"顺手改写"，前面的算法就白算了。系统提示直接写死：

```
你是一位耐心、鼓励型的营养师。你只能改写语气，不得修改、新增、删除任何数字或结论：
输出 120 字以内中文，不要分点。
```

同一份数据，两次输出对比：

```
模板：……这套组合合计 590 kcal、蛋白 55 g，热量偏差 +2%，蛋白达成 153%。
润色：……你这套组合合计 590 kcal、蛋白 55 g，热量只偏差 +2%，蛋白达成 153%，挺不错的，继续加油！
```

逐位比对：1312 / 1804 / 1443 / 91 / 40% / 577 / 36 / 590 / 55 / +2% / 153% —— 11 个数字全部一致。

第三步：不确定的情况要能兜底

把 key 换成一个无效值再请求：

```
code=0，aiReason 为空，reason 仍是模板文案，耗时 0.3s
```

接口不报错、页面不缺内容，只是少了那句"继续加油"。这一点写进了 `prompts/05` 和 README 的限制表：大模型是增强项，不是依赖项。

第四步：去 AI 味

这一条比接 key 麻烦。改了三类东西：

原来	改成	为什么
本章回答一个问题：在「代码主要由 AI 产出」的前提下……	这一章的疑问很具体：代码主要由 AI 产出时，人靠什么保证结果不走样。	设问句 + 长定语是典型的模型腔
几个刻意的设计：蛋白过量要扣分……	几处取舍：蛋白质超过目标 1.3 倍开始扣分……	"刻意的设计"是在自我表扬，读着假
诚实声明： ……伪造的 Prompt 链在评审追问下必然崩塌	说明： 最早的 6 次提交早于归档机制，这里也不补写。	解释"我为什么诚实"本身就是表演

另外把 `prompts/*.md` 的标题统一成「需求 / 拆解 / 取舍 / 接口 / 验证 / 没做的」，去掉「AI 的拆解」「协作形态：DSH Agent 会话」这类标签——标签是给机器看的，记录是给人看的。

复盘

"让 AI 写得像人"最容易走偏的地方，是把它改得更花哨。实际有效的只有三件事：去掉自我评价、去掉设问和排比、把每句话落到具体数字和具体动作上。第三点尤其重要——"处理了并发问题"是空话，"先写菜单再写订单，最坏是少卖而不是超卖"才是人写的。

第二部分 迭代记录

01 · 订单落库

- 日期：2026-10-28
- 怎么做的：我提需求和验收标准，AI 出方案、写代码
- 对应提交：`feat(server): 订单落库 API` → `feat(preview): 结算页对接下单接口` → `feat(admin): 后台订单列表` → `docs(readme): 补充订单 API`

---

1. 需求

> 先补 prompts/ + 订单落库

上下文（会话中已确认）：

- 赛项：2026 第九届传智杯「AI 创新应用挑战赛」个人赛，评审维度中「解题有效性与实用价值」占 25%，要求作品具备完整的产品闭环；
- 现状：`preview/index.html` 的「提交订单」只做了前端校验 + `showPage('Tracking')` 跳转，订单从未落库，跟踪页是一张写死的静态页面；
- 硬约束：服务端是零第三方依赖的原生 Node `http`，数据以 JSON 文件持久化，不能引入数据库或 npm 包。

2. 拆解

把一句话需求拆成三个可独立验收的交付物 + 一个过程性交付物：

#	交付物	验收标准
A	订单持久化接口	`POST /api/orders` 返回订单号；重启服务后订单仍在
B	点餐端闭环	提交订单后，跟踪页显示真实订单号与金额，购物车清空
C	商家端闭环	后台能看到订单，能把状态从「待接单」推到「已完成」
D	过程归档	`prompts/` 目录 + 本文件，与提交一一对应

3. 取舍

决策点	备选	选择与理由
存储	SQLite（`node:sqlite`）/ 单文件 JSON / 内存	单文件 **JSON**。零依赖约束下最稳；`node:sqlite` 在 Node 18 不可用，而 README 承诺 Node ≥ 18；内存方案重启即丢，无法体现"落库"
写入方式	直接 `writeFileSync` / 临时文件 + `rename`	沿用现有 **`writeMenu`** 的原子写。JSON 被写坏 = 全部订单丢失，这个风险不值得省几行代码
金额计算	前端算好传给后端 / 后端重算	后端重算。前端传 `unitPrice` 只作为规格加价后的单价依据，小计、配送费、打包费、合计、总热量全部服务端重算——这是"金额不可被前端篡改"的最小防线
库存	不扣 / 下单扣减	下单扣减。否则后台改库存只是摆设，闭环不成立；库存不足直接 409 拒绝下单
订单状态	只存 pending / 四态流转	四态（待接单 / 制作中 / 已完成 / 已取消），否则后台没有可操作的东西
订单文件入库	提交 `data/orders.json` / 忽略	忽略。提交一个空 JSON 会让每次本地测试产生脏 diff；改为提供 `data/orders.example.json` 说明结构，运行时自动创建

4. 接口

```
POST /api/orders
{ "items":[{ "dishId":"d1", "qty":2, "unitPrice":3200, "specDesc":"份量:加大" }],
  "address":{ "name":"张三","phone":"13800000000","building":"3 号楼","room":"502" },
  "remark":"少冰" }
→ 200 { "code":0, "data":{ "id":"20261028120304-4821", "status":"pending",
                           "subtotal":6400, "deliveryFee":0, "packFee":200, "total":6600 } }
→ 400 参数错误 / 菜品不存在 / 单价低于菜品基础价
→ 409 库存不足
```

- `GET /api/orders?limit=50`：按创建时间倒序返回；
- `PUT /api/orders/:id`：`{ "status": "accepted" | "done" | "canceled" }`，非法流转直接 400。

5. 验证

见提交说明与 `README.md` 第 4 节自检清单，核心三项：

1. `POST /api/orders` 后进程重启，`GET /api/orders` 仍返回该订单；
2. 下单后 `GET /api/menu` 中对应菜品 `stock` 减少，后台刷新可见；
3. 库存置 0 后再下单返回 409，前端 Toast 提示且不跳转。

6. 没做的

- 订单文件容量：当前只做「保留最近 N 条」的截断，未做分页与归档；
- 并发：多个请求同时下单时，`readMenu` → 扣减 → `writeMenu` 之间存在竞态窗口；单机演示场景可接受，生产需加写锁；
- 无鉴权：`GET /api/orders` 任何人可读，含手机号与地址，已在 README「已知限制」中标注。

02 · 后台登录与鉴权

- 日期：2026-10-28
- 怎么做的：我提需求和验收标准，AI 出方案、写代码
- 对应提交：`feat(server)：后台会话与接口鉴权` → `feat(admin)：登录遮罩与退出登录`
- 上游问题：README「已知限制」第 4 条 —— 后台与订单接口无鉴权，订单含手机号与地址

---

1. 需求

> 好的你 1+2 先并行吧

上下文：1 = 技术文档，2 = 再攒 2 条 Prompt 记录（建议做「后台登录与鉴权」和「订单状态驱动跟踪页」）。本条对应前者。

具体要解决的问题（由 AI 从已知限制中提出，人类确认）：

#	问题	期望
1	`PUT /api/dishes/:id` 任何人可改价改库存	必须登录后台才能写
2	`GET /api/orders` 明文返回手机号与地址	必须登录后台才能读
3	`POST /api/upload` 可被任意调用写盘	必须登录后台才能传

2. 拆解

一句话：把「后台」这个角色显式建模出来，而不是靠前端藏按钮。

- 服务端：会话签发 / 校验 / 失效 + 受保护接口清单 + 登录失败限速；
- 后台页：启动先探测会话，未登录弹登录遮罩；接口返回 401 时统一回到登录态；
- 顾客端：不受影响（下单、查自己的订单仍需匿名可用）。

3. 取舍

决策点	备选	选择与理由
会话载体	JWT / HttpOnly Cookie + 服务端 Map / Basic Auth	**Cookie + 内存 Map**。零依赖下 JWT 要自己实现 HMAC 与过期校验，收益不大；Basic Auth 每次请求带密码，浏览器体验差且无法主动登出；内存 Map 重启即失效，正好符合「演示态」定位，代价写进限制说明
Cookie 属性	普通 Cookie / HttpOnly + SameSite	**HttpOnly + SameSite=Lax + Path=/**。HttpOnly 挡 XSS 偷 token，SameSite 挡跨站请求伪造；不过 HTTPS 环境下还应加 `Secure`，本地 http 调试加不了，已记入遗留
口令比对	`===` / 哈希后 `timingSafeEqual`	哈希后定时安全比较。原始字符串比较会因长度/前缀提前返回而泄漏时序信息，`timingSafeEqual` 要求等长，所以先各自 SHA-256
会话续期	固定过期 / 滑动续期	滑动续期（每次命中延长到 8 小时）。演示时不会写着写着被踢出去
暴力破解	不限 / 全局计数 / 按 IP 计数	按 **IP** 计数，5 次失败锁 60 秒。全局计数会被一个人锁死所有人
默认口令	硬编码 / 环境变量	环境变量优先，缺省 **`admin`/`dika2026`**。缺省值写进启动日志与登录框提示，保证评委 `git clone` 后零配置可复现
顾客端订单查询	也要登录 / 凭订单号匿名查	凭订单号匿名查，且手机号脱敏（`136****5678`）。订单号即凭证，评委无需账号也能看到跟踪闭环

4. 接口

```
POST /api/login    { "username":"admin", "password":"dika2026" }
→ 200 { "code":0, "data":{ "user":"admin", "expiresIn":28800000 } }
      Set-Cookie: dika_admin=<48位hex>; HttpOnly; SameSite=Lax; Path=/; Max-Age=28800
→ 401 账号或密码不正确
→ 429 登录失败次数过多，请 N 秒后再试

POST /api/logout        → 200 { "code":0 }，同时清除 Cookie
GET  /api/session       → 200 { "code":0, "data":{ "user":"admin" } } / 401

GET  /api/orders/:id    → 200 公开视图（手机号脱敏、含 events 事件链）/ 404
```

受保护清单：`PUT /api/dishes/:id`、`POST /api/upload`、`GET /api/orders`、`PUT /api/orders/:id`。

公开清单：`GET /api/menu`、`POST /api/orders`、`GET /api/orders/:id`。

5. 验证

用例	结果
未登录 `GET /api/orders`	401
未登录 `PUT /api/dishes/d1`	401
未登录 `POST /api/upload`	401
错误口令	401 账号或密码不正确
正确口令	200，`Set-Cookie` 写入 cookie jar
带 Cookie 访问 session / orders / 改菜品	200 / 200 / 200
登出后再访问 `/api/orders`	401
公开下线（无 Cookie）	200，订单正常创建

6. 没做的

- 内存会话：进程重启后全部登录态失效（评审演示需重新登录一次，属预期）；
- 未加 `Secure` 属性（本地 http 环境无法设置），部署 HTTPS 时应补；
- 登录限速只按 IP，NAT 后多人共享出口 IP 时可能互相影响；
- 没有角色区分：目前只有一个 `admin` 角色，后续若要区分店长/店员需扩展 `sessions` 里的角色字段。

03 · 订单状态驱动的跟踪页

- 日期：2026-10-28
- 怎么做的：我提需求和验收标准，AI 出方案、写代码
- 对应提交：`feat(server)：订单事件链` → `feat(preview)：跟踪页轮询真实状态`
- 上游问题：README「已知限制」第 2 条 —— 订单已落库，但跟踪页进度条仍是写死的演示态

---

1. 需求

> 好的你 1+2 先并行吧

上下文：补第 2 条 Prompt 记录，把「订单状态驱动跟踪页」做成真实闭环——商家在后台点「接单 / 完成」，顾客端跟踪页应当自己变化。

改造前的具体问题（读代码确认）：

位置	问题
`preview/index.html` 跟踪页	进度条 4 个节点是写死的 `class="done"/"current"`，文案「商家已接单 / 预计 12:35 送达」是常量
时间轴	4 条记录时间固定 `12:02 / 12:03 / --:--`，与真实订单无关
骑手卡片	「配送小哥 王师傅 188****5678」是虚构内容，项目并没有配送体系
数据	订单只有 `status` 与 `updatedAt`，没有状态变更历史，无法还原时间轴

2. 拆解

- 数据层：订单增加 `events：[{ status，at }]` 事件链，创建时写入 `pending`，后台每次改状态追加一条；
- 接口层：新增公开的 `GET /api/orders/:id`（凭订单号查询，手机号脱敏），返回事件链；
- 展示层：跟踪页按 `status` 映射节点状态、按 `events` 渲染时间轴；进入页面后每 5 秒轮询一次，到终态停止；
- 诚实性：删掉虚构的骑手卡片与「预计送达时间」——没有的数据不假装有。

3. 取舍

决策点	备选	选择与理由
状态推进通知	前端轮询 / SSE / WebSocket	**5** 秒轮询。零依赖约束下 SSE 要额外维护长连接与断线重连，WebSocket 在原生 `http` 上要自己实现握手与帧解析；订单状态变化是低频事件，轮询成本可忽略
轮询终点	一直轮询 / 到终态停止	**`done` / `canceled`** 停止，并在离开跟踪页时清掉定时器，避免后台标签页长期空转
时间轴数据	用 `updatedAt` 推算 / 落事件链	落 **`events`**。只有一个 `updatedAt` 时，第二次改状态会覆盖第一次的时间，时间轴必然错位
事件顺序	正序 / 倒序	倒序（最新在最上），符合外卖类 App 的心智；最新一条加粗
订单查询鉴权	与后台同权限 / 凭订单号匿名	匿名。顾客没有账号；手机号脱敏后，泄漏面仅剩「楼栋+门牌」，与外卖平台取餐码同一级别
取消态展示	仍显示 4 节点 / 隐藏进度条	隐藏进度条，顶部状态文案改为「订单已取消」。已完成/已取消是互斥终态，继续画进度条会造成误解
轮询容错	失败弹 Toast / 静默	静默忽略，下一次轮询再试。网络抖动时弹窗会打断用户，而跟踪页本身不是强交互页面

4. 状态映射

订单状态	顶部文案	进度节点	轮询
`pending`	等待商家接单	已提交 √ / 已接单（当前） / 制作中 / 已完成	继续
`accepted`	商家已接单，正在制作	已提交 √ / 已接单 √ / 制作中（当前） / 已完成	继续
`done`	订单已完成	全部打勾	停止
`canceled`	订单已取消	隐藏进度条，时间轴保留	停止

5. 验证

用例	结果
下单后响应含 `events`	1 条 `pending`
匿名 `GET /api/orders/:id`	200，`phone=136****5678`，`events=1`
后台推 `accepted`	200，事件链变为 `pending → accepted`
再次匿名查询	`status=accepted`，`events=2`
跟踪页渲染（headless 截图）	顶部「商家已接单，正在制作」；进度条第 3 节点高亮；时间轴 14:31 接单 / 14:30 下单
虚构内容清理	骑手卡片、「预计 12:35 送达」、写死时间轴均已移除

6. 没做的

- 轮询间隔固定 5 秒，未做「页面隐藏时暂停」（`visibilitychange`），后台标签页仍会每 5 秒请求一次；
- 订单号是「时间戳 + 4 位随机」，理论可枚举；演示场景可接受，生产应改为足够熵的随机串或加签名；

- 没有状态回滚：done 之后仍可被 PUT 改成 canceled，后续应加状态机白名单（当前只挡了改回 pending）。

04 · 小程序端与双端架构

- 日期：2026-10-28
- 怎么做的：我提需求和验收标准，AI 出方案、写代码
- 对应提交：`feat(miniprogram)：新增微信小程序生产端` → `docs(prompts)：04` → `docs：README/技术文档同步`
- 上游问题：README「已知限制」第 1 条 —— 仓库名带 `miniprogram`，但主体只有 H5

---

1. 需求

> 我想要做的是可以上架微信小程序端的 做 web 版本是为了方便在网页端查看 所以就是相当于做了两套代码 一套代码是实际应用的一套是方便展示的 你根据我的意思补充完善一下这个项目代码

随消息附了一张技术栈截图，期望：微信原生小程序（不用 uni-app / Taro / mpvue）、TypeScript strict、WXML + WXSS、运行时零依赖。

这句话其实是一次定位纠正：此前我把 H5 当成了主体、小程序当成"以后再补"，而真实的产品意图是反过来——小程序是交付物，**H5** 只是演示壳。

2. 拆解

- 重新划分两层：`miniprogram/`（生产端，上架用）+ `preview/`、`admin/`（展示与运营），`server/` 两端共用；
- 小程序端做一个完整闭环，不做半成品：点餐（含规格弹层）→ 确认订单（地址/费用/备注）→ 订单跟踪（状态 + 事件时间轴）；
- 字段与 **Web** 端严格对齐，让"同一套接口"不是口号：规格加价用 `deltaPrice`、计费规则取 `menu.fees`、金额以「分」传输；
- 把"能不能上架"拆成代码条件与资质条件，后者写进 README 与文档，不假装技术能解决一切。

3. 取舍

决策点	备选	选择与理由
跨端框架	uni-app / Taro / 原生	原生。只有 3 个页面，框架的编译产物、调试链路与版本兼容成本都高于收益；且本项目要的是"微信里真的能用"，不是"多端发布"
语言	JS / TypeScript strict	**TS strict**。在没有运行时测试的前提下，类型是唯一能在提交前挡下 `deltaPrice` 写成 `delta` 这类错误的闸门；开发者工具原生编译 TS，不需要额外构建
状态管理	mobx-miniprogram / 模块级单例	模块级单例。状态只有「一车商品」，observable 那套反而增加心智负担；页面 `onShow` 主动 `syncCart()`
规格默认值	空选 → 禁用按钮 / 默认选中每组第一项	默认选中。必选组永远有值，用户不必先点一遍才能看到价格
购物车持久化	存 Storage / 不存	不存。跨次启动带着上次没下单的商品，比"丢了再选"更糟
收货地址	不存 / Storage	存。地址几乎不变，重复填写是主要摩擦点
状态同步	WebSocket / SSE / 轮询	**5** 秒轮询，与 H5 保持一致；订单状态是低频事件，两端逻辑相同便于讲清楚

4. 接口与字段对齐核对

环节	字段	核对结果
菜单	`dishes[].specs[].options[].deltaPrice`	✅ 从 `data/menu.json` 实际读取得知（不是文档里写的 `delta`）
计费	`fees.deliveryFee / packFee / freeDeliveryThreshold`	✅ 三字段齐全，页面按 `subtotal >= freeDeliveryThreshold` 判免配送
下单	`POST /api/orders { items[{dishId,qty,unitPrice,specDesc}], address{name,phone,building,room}, remark }`	✅ 与服务端校验一致：`unitPrice = 基础价 + Σdelta`，满足「单价 ≥ 基础价」
查询	`GET /api/orders/:id → { status, events[], total, itemCount, address.phone(脱敏) }`	✅ 跟踪页用 `events` 渲染时间轴

> 这次核对发现一个真实问题：**README** 的规格示例写的是 **`"delta": 0`**，而数据里是 **`deltaPrice`**。小程序端写完后回头把 README 示例一并改正——这类"文档与数据打架"的坑，正是双端对齐时最容易漏掉的。

5. 验证

用例	结果
`npx tsc --noEmit`（strict + noUnusedLocals）	✅ 0 error
依赖审计	运行时依赖 0；开发依赖仅 `typescript`、`miniprogram-api-typings`
文件清单	25 个（含 `project.config.json` / `tsconfig.json` / 三页 12 个文件）
接口字段	逐项对照 `data/menu.json` 与 `server.js`，见上表

6. 没做的

- 本机没有微信开发者工具，UI 未经模拟器/真机验证，只做了 TypeScript 静态检查与接口契约核对——这一条必须写清楚，不能把"编译通过"说成"跑通了"；
- 未接 `wx.login`：顾客仍以订单号为凭证，订单不归属到具体用户；
- 未接微信支付：当前是线下/到店结算；

- 购物车未持久化（刻意为之，见取舍表）；
- 未做分享卡片、订阅消息、骨架屏等小程序侧体验细节。

7. 上架前要准备什么

HTTPS 域名 → ICP 备案 → 服务器域名白名单 → 餐饮类目资质（食品经营许可证）→ 隐私保护指引配置。这五项不解决，代码再干净也提交不了审核。

05 · AI 个性化减脂套餐推荐

- 日期：2026-10-28
- 怎么做的：我提需求和验收标准，AI 出方案、写代码
- 对应提交：`feat(recommend)：推荐引擎与接口` → `feat(ui)：H5/小程序 AI 定制页` → `docs：创新点与文档同步`
- 上游需求：把「内置 AI 个性化减脂套餐推荐，结合用户身体指标给出适配餐品」从一句话变成可点开的功能

---

1. 需求

> 本项目为减脂餐创业商家打造轻量化点餐小程序……业务层面：面向小微轻食商家，提供低成本数字化接单方案，内置 AI 个性化减脂套餐推荐，结合用户身体指标给出适配餐品，同时双前端架构兼顾真实业务落地与评审演示便捷性。

拆出来是一个明确的功能：用户输入身体指标 → 系统给出这一餐该吃什么 → 能直接下单。

2. 拆解

层	交付物	验收标准
数据	菜品补 `protein`（克）	28 道菜全部有值，并声明为估算
算法	`server/recommender.js`	同输入同输出；每步可解释（BMR/TDEE/目标/偏差）
接口	`GET /api/recommend/options`、`POST /api/recommend`	参数非法 400；全售罄 409
展示	H5 定制页、小程序定制页	表单 → 结果 → 一键加购
说明	README / 技术文档 / 本记录	讲得清「为什么不是随便调个大模型」

3. 取舍

决策点	备选	选择与理由
推荐由谁算	直接调大模型生成套餐 / 本地营养学模型 + 组合搜索	本地模型。推荐结果要能复现、要能解释、要零成本——大模型每次输出都可能不同，评审问「为什么是这份」时答不上来。大模型只用于文案增强（见下）
大模型角色	不用 / 用来算 / 用来润色	只润色，默认关闭：配置 `LLM_API_KEY` 才调用，Prompt 里写死「只改语气，不得改动任何数字」，失败自动回落模板文案。实测见第 5 节
推荐粒度	全天三餐 / 单餐	单餐（全天 **40%**）。用户是在点一餐，不是在做营养规划
目标函数	只按热量 / 热量 + 蛋白 + 价格	三项加权：蛋白达成 0.5、热量偏差 -0.35、价格 0.15；另加蛋白过量惩罚（超过目标 1.3 倍开始扣分），否则算法会无脑堆鸡胸肉
预算无解	返回失败 / 忽略预算 / 给最便宜的合规组合	最便宜且满足营养下限（蛋白 **≥60%**、热量 **≥70%**），同时在结果里标注 `budgetRelaxed`，让用户知道放宽了
营养数据	不写 / 精确值 / 估算值	估算值 + 数据文件里写明 **`nutritionNote`**。伪造"实验室级"数据比承认估算更不诚实

4. 计算链路

```
身体指标 → Mifflin-St Jeor
  BMR = 10×体重 + 6.25×身高 − 5×年龄 + (男 5 / 女 −161)
→ TDEE = BMR × 活动系数(1.2 / 1.375 / 1.55 / 1.725)
→ 每日目标 = TDEE × 目标系数(减脂 0.8 / 维持 1.0 / 增肌 1.1)，并设下限 1200/1400/1600
→ 蛋白目标 = 体重 × (1.6 / 1.2 / 1.8) g
→ 单餐预算 = 目标 × 40%
→ 组合搜索：主菜(1) × 加料(0-1) × 汤/果汁(0-1) × 甜点(0-1)，约 1.5k 组合
→ 目标函数打分，取最高
```

5. 验证

用例	输入	结果
手算核对	女 28 岁 165cm 58kg 轻度	BMR 1310 / TDEE 1801，与手算 1310.25 / 1801.6 一致
正常推荐	同上，减脂	鸡胸肉藜麦沙拉 + 额外鸡胸肉 + 番茄时蔬汤 = 530 kcal / 59 g / ¥57，热量偏差 −8%
素食过滤	男 72kg，`vegetarian`	三项全部为素食菜品（牛油果藜麦能量碗 / 南瓜浓汤 / 抹茶豆腐慕斯）
预算过低	女 52kg，预算 ¥10	`budgetRelaxed=true`，退化为最便宜的合规组合（全麦鸡胸三明治 ¥26）
确定性	同参数请求两次	菜品 id 完全一致
非法参数	`age: "x"`	400 + 中文提示
大模型润色（真实调用）	DeepSeek，`useAI: true`	返回 `aiReason`，11 个数字（1312 / 1804 / 1443 / 91 / 40% / 577 / 36 / 590 / 55 / +2% / 153%）与模板完全一致，只把语气改得更口语
坏 Key 兜底	传一个无效 Key	0.3 秒返回，`aiReason` 为空，`reason` 仍是模板文案，接口 200 不受影响

6. 没做的

- 蛋白为估算值，未做碳水/脂肪/膳食纤维，也未做过敏原与忌口过滤；
- 未利用历史订单做协同过滤（现在是个体化，不是"越用越准"）；
- 大模型润色已用 DeepSeek 实测，但没有做多模型对比与成本核算，也没有对生成文案做敏感词过滤；
- 小程序端同 H5 一致地只做静态类型检查，UI 未在模拟器验证。

06 · 运行日志与 CI

- 日期：2026-10-29
- 对应提交：`feat(log)`：结构化日志 → `ci`：冒烟工作流 → `docs`
- 上游问题：出问题时只能靠复现，推荐为什么选这三样、订单金额怎么算出来的，全靠猜

---

1. 需求

> 继续完善这个程序的日志，就是包括 AI 推荐计算热量什么的

要的不是 `console.log('here')`，而是能回答三个问题：这单为什么是这个价、这套餐为什么这么推、这次请求为什么慢。

2. 拆解

交付	验收
结构化日志模块	一行一条 JSON，可 `jq` 直接过滤
请求日志	只有 `/api/*`，带状态码与耗时
推荐日志	指标 → BMR/TDEE → 目标 → 候选数 → 选中组合 → 偏差，一条看全
订单日志	金额拆分、热量、菜品；状态流转记 from/to
认证日志	成功/失败/锁定，绝不落密码
脱敏	手机号、口令、token 落盘前替换
CI	语法 + 类型 + 接口冒烟，push 自动跑

3. 取舍

决策点	备选	选择与理由
日志格式	自由文本 / 结构化 JSON	**JSON Lines**。文本日志想统计"推荐平均耗时"只能写正则，JSON 直接 `jq` 聚合
依赖	winston / pino	零依赖，与项目一贯的约束一致；功能只要"分级 + 落盘 + 滚动"，自己写 90 行足够
落盘策略	每请求一个文件 / 单文件不滚 / 单文件 + 按大小滚	单文件 + **5MB** 滚动保留一份。小服务器最怕日志把磁盘写满
静态资源	全记 / 不记	不记。图片和页面会把真正有用的 API 日志淹掉
脱敏位置	打印时 / 写入前	写入前统一 **sanitize**，控制台和文件都不会漏；靠"记得别打手机号"迟早出事
推荐耗时	只记总耗时	另记候选组合数。菜单从 28 道加到 60 道时，候选数会先于耗时暴露问题
CI 范围	只跑语法 / 加类型与冒烟	语法 + 类型 + 冒烟。这个项目没有测试框架，冒烟是最低成本的行为回归

4. 日志事件

```
{"ts":"2026-10-29T14:14:37.861Z","level":"info","event":"recommend",
 "profile":{"gender":"female","age":26,"height":165,"weight":58,"activity":"light","goal":"cut",
            "bmr":1320,"tdee":1815,"targetCalorie":1452,"proteinTarget":93},
 "mealTarget":581,"mealProteinTarget":37,"candidates":704,"elapsedMs":6,
 "picked":["d9","d26","d18"],
 "totals":{"calorie":590,"protein":55,"price":5100},
 "match":{"calorieDiffPct":2,"proteinPct":149},"budgetRelaxed":false,"vegetarian":false}
```

这条记录把"为什么是这套"完整还原了：指标 → 代谢 → 目标 → 预算 → 搜了多少组合 → 选中哪三道 → 偏差多少。

5. 验证

用例	结果
启动	`server.start` 记录端口、管理员账号、日志路径
`GET /api/menu`	`http` 事件，200，6ms
`POST /api/recommend`	`recommend` 事件，候选 704 组、耗时 6ms、选中 d9/d26/d18
`POST /api/orders`	`order.create`，金额拆分 3200+500+200=3900，热量 320
错误口令	`auth.login ok:false`，fails 计数
正确口令 + 访问订单	`ok:true`，随后 `GET /api/orders` 200
脱敏单测	`{phone,password}` → `{"phone":"*","password":"*"}`

6. 没做的

- 没有集中式日志服务，也没有做日志查询接口（会把内部信息暴露给前端）；
- 没有对日志做结构化采样，高并发时会全量写盘；
- 推荐日志里的 `picked` 只存菜品 id，没存当时的单价快照，菜单改价后回溯会有偏差；
- CI 只在 push/PR 时跑，没有定时巡检。

07 · 一句话点餐

- 日期：2026-10-29
- 对应提交：`feat(nl)`：自然语言点餐 → `docs`：文档与材料同步
- 上游问题：赛道的 10 个推荐方向里没有餐饮，作品容易被看成"普通点餐系统"；推荐模块又被定位成"AI 只润色"，AI 含量不足以支撑 20% 的「方法创新性与意图控制」

---

1. 需求

> 你全部都做然后上传 GitHub —— 让这个项目从"点餐系统 + 一个小推荐"变成"能听懂人话的点餐助手"

本来的交互是：用户翻类目、看菜品、自己算热量。改成：说一句话，系统把话拆成条件，再筛。

2. 拆解

交付	验收
意图解析接口	一句话 → 结构化条件；模型失败时正则兜底
条件筛选	热量/蛋白/价格/素食/忌口/关键词，六个维度都能生效
筛不到时放宽	逐级去掉最软的条件，并把"放宽了什么"回传
双端入口	H5 菜单页顶部输入条；小程序同款
可复现	同一句话 + 同一菜单 = 同一结果，`temperature=0` 且筛选在本地

3. 取舍

决策点	备选	选择与理由
谁来解析	让模型直接推荐菜品 / 模型只解析条件	只解析。让模型点菜，它每次给你的组合都可能不同，评审问"为什么是这三道"就答不上；解析成条件后，筛选在本地跑，结果可复算
解析失败	直接报错 / 猜一个默认条件 / 正则兜底	正则兜底。"500千卡以内""不要香菜""预算40"这些说法用正则就能覆盖大半，配置没到位或网络抖动时也不至于功能不可用
模型输出校验	信任模型返回 / 逐字段校验	逐字段校验：数值做范围截断，数组限制长度与条数，非法值直接丢弃，不做猜测
筛不到怎么办	返回空 / 自动放宽 / 自动推荐别的	逐级放宽并回传。关键词 → 忌口 → 蛋白 → 价格 → 热量 → 素食，每去掉一项就记一笔，前端明说"已放宽 xxx"，不假装命中
条件怎么展示	只给结果 / 结果 + 条件标签	标签化。用户能一眼看出模型理解对不对（比如把"高蛋白"理解成没条件），这是意图对齐最便宜的验证方式
双端一致性	只做 H5	两端都做。小程序是生产端，H5 是演示端，功能缺一个都说不通

4. 接口

```
POST /api/nl-order
{ "text": "我想吃500千卡以内、高蛋白、不要香菜，预算40" }

→ 200
{
  "source": "llm",              // llm | llm+fallback | fallback | none
  "conditions": { "maxCalorie": 500, "maxPrice": 40, "excludeKeywords": ["香菜"], ... },
  "tags": ["热量 ≤ 500 kcal", "价格 ≤ ¥40", "不要香菜"],
  "dropped": [],                // 被放宽掉的条件
  "dishes": [ { "id": "d13", "name": "香煎鸡胸配时蔬", "calorie": 330, "protein": 33, ... } ]
}
```

5. 验证

用例	结果
"500 千卡以内、高蛋白、不要香菜，预算 40"	`source=llm`，条件解析为热量 ≤500 / 价格 ≤40 / 不要香菜，命中 8 道，前三是香煎鸡胸(330kcal/33g)、鸡胸藜麦沙拉(320/32)、清蒸龙利鱼(260/28)
"来点素的，热量别超过 300"	只筛选素食，命中 7 道，首位抹茶豆腐慕斯 190kcal
"想吃鸡胸，蛋白 25 克以上"	命中 3 道，蛋白全部 ≥25g
"预算 30，来碗热的"	关键词"热"无匹配 → 自动放宽，`dropped=["想吃热"]`，返回价格 ≤30 的 8 道
未配置 Key 时的同一句话	`source=fallback`，正则解析出热量/价格/忌口，结果与模型版接近
小程序端 `tsc --noEmit`	0 error

6. 没做的

- 不做多轮追问（"再来点清淡的"这类依赖上下文的说法，当前每次独立解析）；
- 不做方言与错别字纠正，正则兜底只覆盖常见句式；
- 模型解析结果没有做缓存，同一句话重复请求会重复计费；
- 没接菜单之外的知识（比如"适合增肌的"这种需要营养学推理的说法，只能落到关键词）。

08 · 经营看板

- 日期：2026-10-29
- 对应提交：`feat(stats)`：经营看板、`docs`：看板章节与材料同步
- 上游问题：赛道推荐方向 6 的描述是「基于标准数据格式的智能分析系统 + 动态图表大盘 + 大模型对长尾数据做趣味解读」。原项目只有商品与订单管理，没有"看数据"这一面，对位是虚的。

---

1. 需求

商家想知道今天卖得怎么样：几单、多少钱、什么卖得好、什么时候忙、顾客点的热量高不高。原来这些只能一条条翻订单。

2. 拆解

交付	验收
聚合接口	今日订单数（含待接/完成/取消）、营业额、客单价、24 小时分布、热销 Top5、客单热量分桶、库存告急
看板界面	后台一个区块，指标卡 + 时段柱状 + 热销条 + 热量分布条
AI 解读	一键让模型读今天的数字，给 3 条建议；没有 Key 时用模板
数字可信	模型只能复述给定的数字，不新增、不推测

3. 取舍

决策点	备选	选择与理由
图表库	引 ECharts / 手写 canvas / 纯 CSS 条	纯 **CSS** 条。项目零依赖是卖点，为几张图引一个 300KB 的库不值；这里的数据量级也不需要交互图表
统计口径	全部订单 / 排除已取消	排除已取消。取消单不该进营业额与客单价，否则数据虚高
热量统计	按单品累加 / 按订单合计后分桶	按订单合计分桶。顾客关心的是"这一餐多少卡"，不是单个菜
AI 角色	让模型自己算 / 数字算好只让它解读	只解读。模型不会数数，让它算必然出错；把结构化数字喂给它，只要求写建议
无 Key 时	报错 / 隐藏入口 / 模板文案	模板文案。模板用同一份数字拼句子，逻辑与模型版一致，只是不好听

4. 接口

```
GET  /api/stats            （需登录）
POST /api/stats/insight    （需登录）
```

`GET /api/stats` 返回今日聚合：`orders{total,active,done,pending,canceled}`、`revenue`、`avgTicket`、`hours[24]`、`topDishes[5]`、`calorie{avg,buckets[4]}`、`inventory{low,soldOut}`、`peakHour`。

`POST /api/stats/insight` 返回 `{ text, source }`，`source` 为 `llm` 或 `template`。

5. 验证

用 7 笔测试订单（含 1 笔库存不足被 409 拒绝）跑通：

检查	结果
聚合准确性	6 单、营业额 ¥418、客单价 ¥70、峰值 19 点，与手工核算一致
热销排序	鸡胸肉藜麦沙拉×4、烟熏三文鱼沙拉×3、照烧鸡腿糙米碗×2…
热量分桶	300–500 两单、>700 四单，与订单热量对得上
AI 解读	`source=llm`，输出 3 条建议，数字全部来自接口（6 单、售罄 4 项、19 点、¥70）
未配 Key	`source=template`，输出同结构的一句话摘要
权限	未登录访问 `/api/stats` 返回 401

6. 没做的

- 没做多日趋势与同比（需要更长时间的数据积累，两天造不出来）；
- 没做导出 CSV / Excel；
- 图表固定为"今日 + 8–22 点"，没有做时间范围切换；
- 模型解读没有做缓存，同一份数据重复点会重复计费。

第三部分 归档说明

prompts/ · 迭代记录

这里放每次功能迭代的过程记录。赛项要求「至少 3 个核心功能迭代的完整 Prompt 链」，所以每条都写清楚：当时提了什么要求、AI 给了什么方案、我怎么选、最后怎么验证的。

写法约定

- 一次功能迭代一个文件，命名 `NN-功能名.md`，编号跟提交顺序一致；
- 提交信息里带上编号，方便对照；
- 只写真实发生过的事，没做的就写没做；
- token、密钥、手机号一律打码。

每条包含

1. 需求：当时的原话或关键句
2. 上下文：代码到什么状态、有什么硬限制
3. 拆解：要交付哪几样，怎么算完成
4. 取舍：备选方案和放弃的理由（评审最看这段）
5. 接口 / 链路：改动的具体形状
6. 验证：跑过什么、结果如何
7. 没做的：这次没覆盖的部分

索引

编号	功能	对应提交
01	订单落库	`feat(server)`：订单落库 `API` 起共四次
02	后台登录与鉴权	`feat(server)`：后台会话与接口鉴权、`feat(admin)`：登录遮罩
03	订单状态驱动跟踪页	`feat(server)`：订单事件链、`feat(preview)`：跟踪页轮询真实状态
04	小程序端与双端架构	`feat(miniprogram)`：微信小程序生产端
05	AI 减脂套餐推荐	`feat(recommend)`、`feat(ui)` 两次提交
06	运行日志与 CI	`feat(log)`、`ci`：两次提交
07	一句话点餐	`feat(nl)`、`docs`：两次提交

早于这套记录的部分

仓库最早的 6 次提交是存量代码按模块导入，那时还没有这套记录，所以没有对应条目，也不补写。

对话快照（snapshots/）

赛项要求提交"AI 对话历史导出的 Prompt 历史快照"，所以另开一个目录放带真实报错与命令的对话纪要：

文件	内容
`snapshots/01-建仓与推送排障.md`	建仓 400（GBK 编码）、账号 404、push 被重置，四个卡点怎么定位
`snapshots/02-推荐引擎调参.md`	第一版蛋白 173% → 加过量惩罚 → 预算无解时退化策略
`snapshots/03-大模型润色的数字约束.md`	系统提示怎么防模型改数字、坏 Key 兜底、去 AI 味的具体改法

这些是纪要（节选），不是逐字转录；原始对话截图/录屏可另行补充。

附录　对话过程截图

以下截图取自开发时与 AI 的会话界面，按时间顺序排列，用于佐证 prompts/ 下三份对话快照记录的过程真实发生。截图未做美化，保留当时的窗口与侧栏。



图 A-1　起始对话：确认赛项与规则，明确要交的是哪些材料

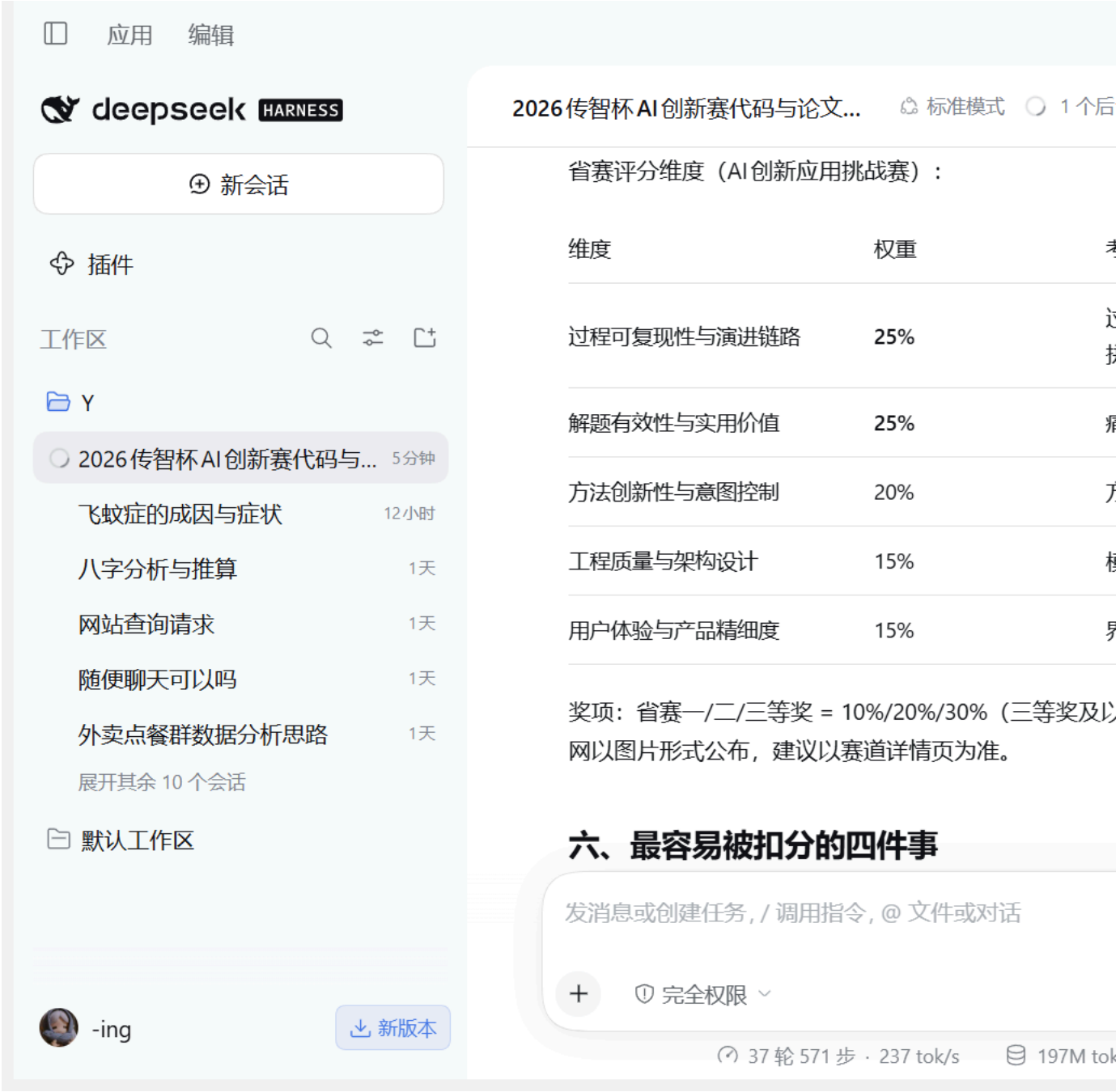


图 A-2　评分维度：过程可复现 25%、解题有效 25%、方法创新 20%，决定了后续所有取舍

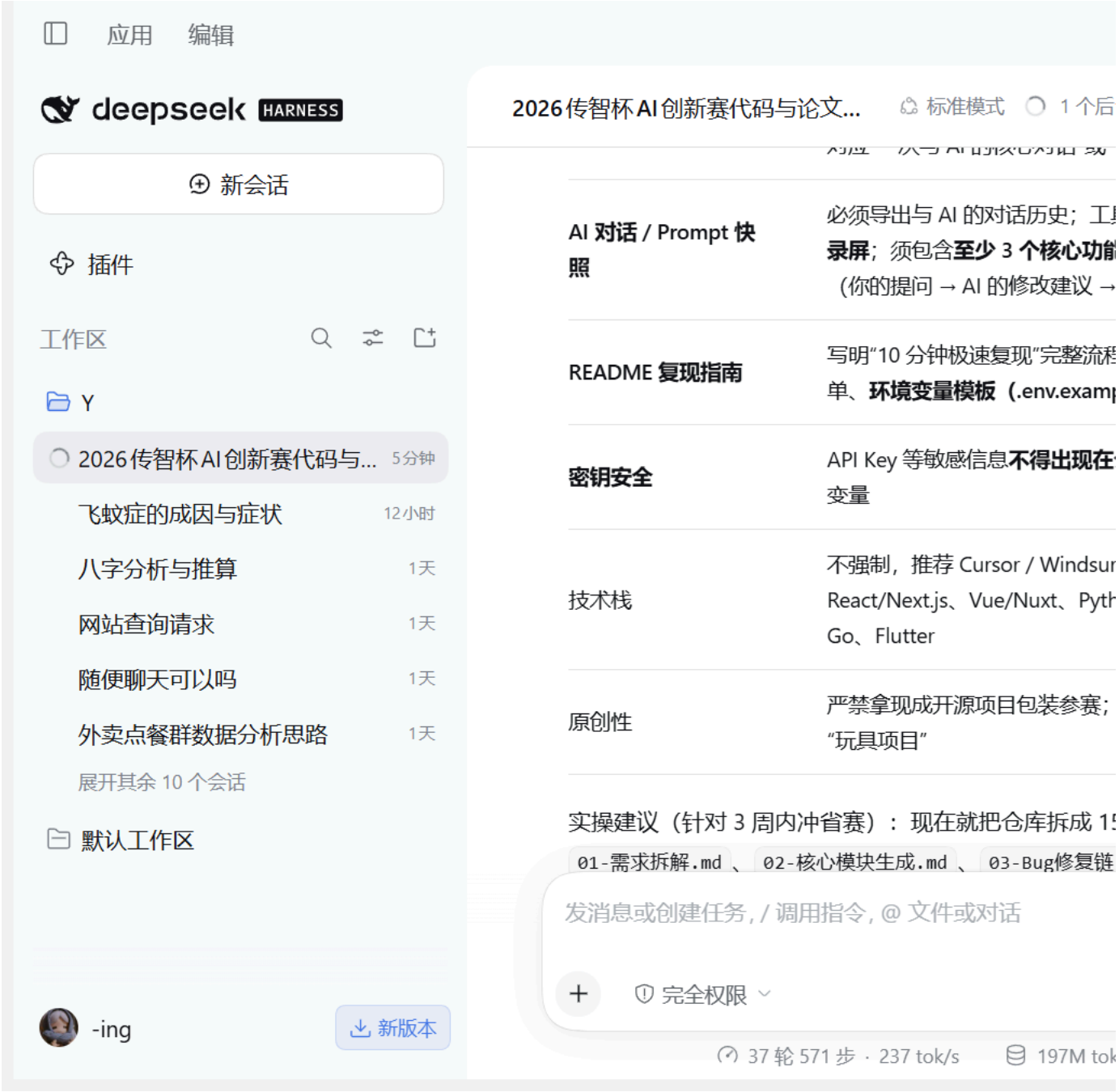


图 A-3　材料要求：Prompt 快照、README 复现指南、密钥安全，逐条对到仓库里的文件

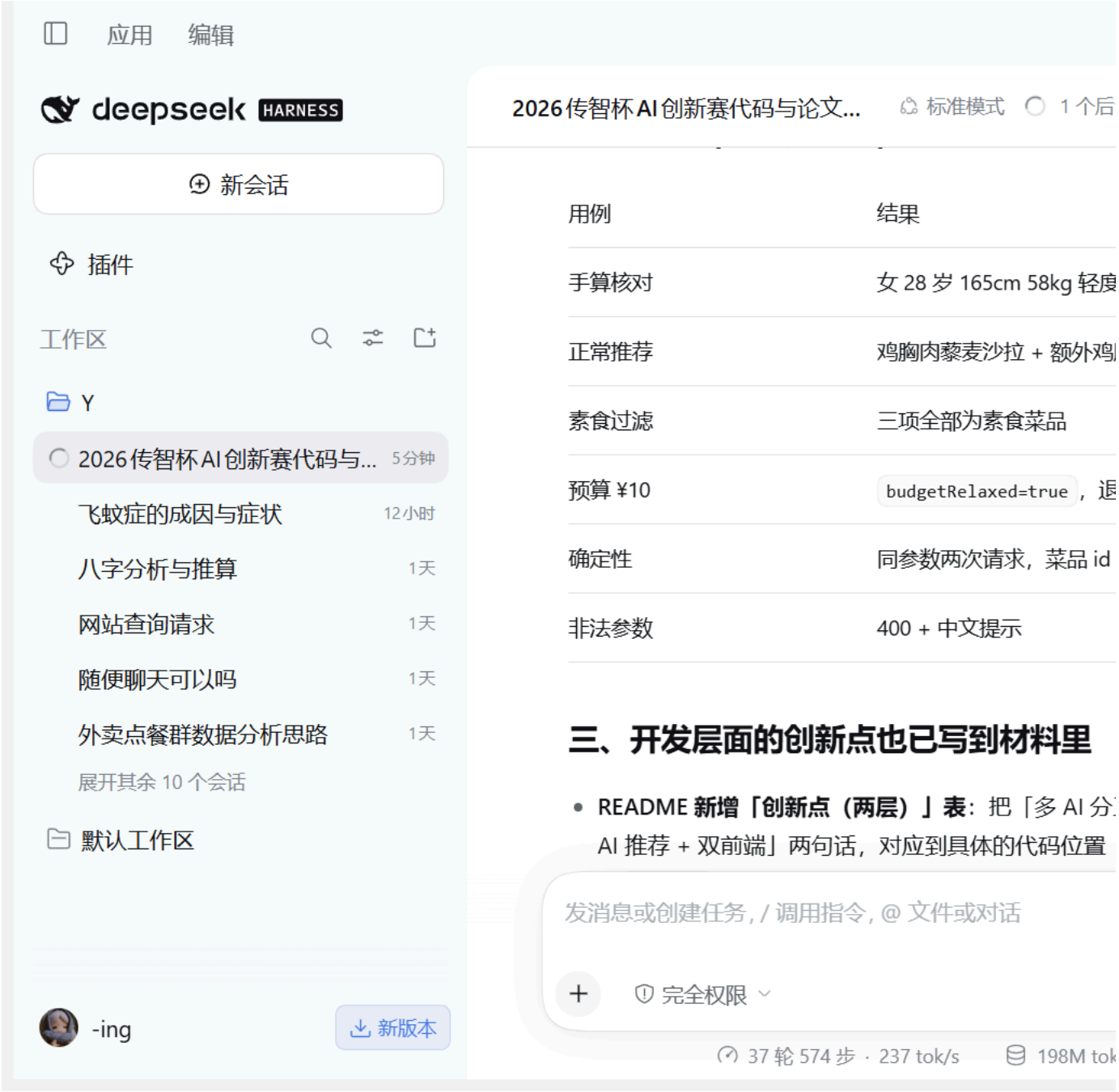


图 A-4　推荐引擎核对：手算 BMR / TDEE 与程序结果一致，同参数两次请求菜品完全相同

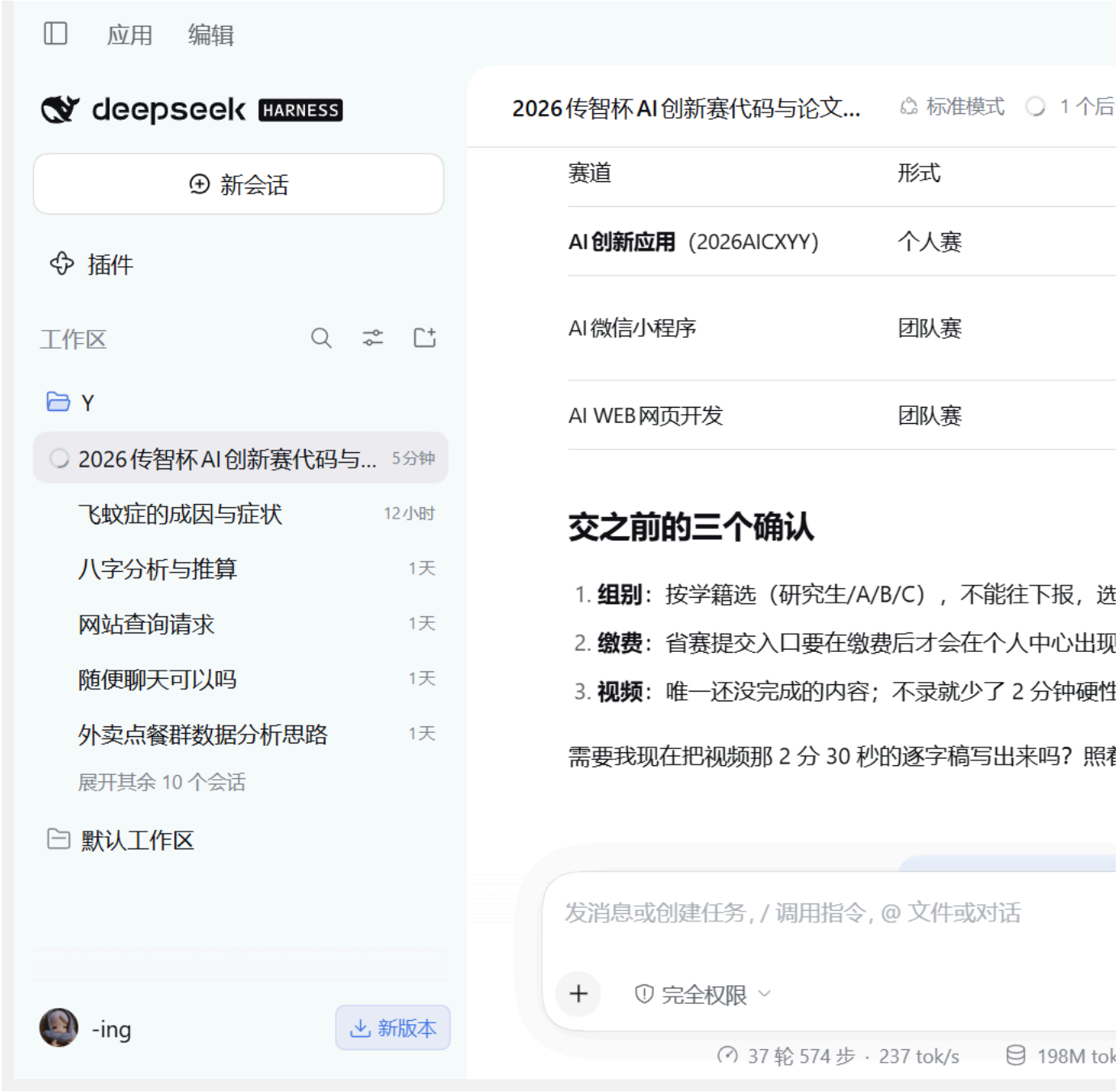


图 A-5　赛道对比：个人赛与两个团队赛的取舍，以及交件前的三个确认



图 A-6　仓库核验：本地与远程同提交、工作区干净、匿名可读，这一屏是提交前的最后检查